🚀React Day 10

UseAction State Hook

```jsx
import React, { useActionState } from 'react';

const UseAction = () => {
  const handleSubmit = async (previousData, formData) => {
    const name = formData.get('name');
    const password = formData.get('password');

    await new Promise(res => setTimeout(res, 2000)); // simulate delay
    console.log("function reached");

    if (name && password) {
      return {
        success: true,
        message: '✅ Data submitted successfully!',
        name,
        password
      };
    } else {
      return {
        success: false,
        message: '❌ Failed to submit data. Please fill all fields.'
      };
    }
  };

  const [data, action, pending] = useActionState(handleSubmit, undefined);

  return (
    <div style={{ padding: '20px', fontFamily: 'Arial' }}>
      <form action={action}>
        <h3>🚀 useActionState Example</h3>

        <input type="text" placeholder='Enter name' name='name' />
        <br /><br />
        <input type="password" placeholder='Enter password' name='password' />
        <br /><br />

        <button type="submit" disabled={pending}>
          {pending ? 'Submitting...' : 'Submit'}
        </button>

        <br /><br />

        {data && (
          <p style={{ color: data.success ? 'green' : 'red' }}>
            {data.message}
          </p>
        )}
      </form>

      {/* 👇 Submitted Data Display */}
      {data?.success && (
        <div style={{ marginTop: '20px', borderTop: '1px solid #ccc', paddingTop: '10px' }}>
          <h4>📋 Submitted Details:</h4>
          <p><strong>Name:</strong> {data.name}</p>
          <p><strong>Password:</strong> {data.password}</p>
        </div>
      )}
    </div>
  );
```

useActionState is a **new hook introduced in React 19** that helps you **handle form submissions** and **manage UI state** (like pending, result, etc.) **on the client side** in a **declarative** way.

⚙️ Syntax:

const [state, actionFunction, pending] = useActionState(handlerFunction, initialState);

UseId Hook

In React, the useId hook is used to **generate unique IDs** that are consistent between the server and the client (important for SSR - Server Side Rendering). This is useful when you need to associate form elements like <label> with <input>, or when generating unique keys for accessibility purposes.

```jsx
import React, { useId, useState } from 'react';

const UseId = () => {
  const nameId = useId();
  const emailId = useId();
  const passwordId = useId();

  const [formData, setFormData] = useState({
    name: '',
    email: '',
    password: '',
  });

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value,
    });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Successfully completed:', formData);
  };

  return (
    <div>
      <h3>Use Id Hook</h3>
      <div>
        <form onSubmit={handleSubmit}>
          <div style={{ marginBottom: '15px' }}>
            <label htmlFor={nameId}>Name:</label><br />
            <input
              id={nameId}
              name="name"
              type="text"
              value={formData.name}
              onChange={handleChange}
              placeholder="Enter your name"
              required
            />
          </div>

          <div style={{ marginBottom: '15px' }}>
            <label htmlFor={emailId}>Email:</label><br />
            <input
              id={emailId}
              name="email"
              type="email"
              value={formData.email}
              onChange={handleChange}
              placeholder="Enter your email"
              required
```

```
      />
    </div>

    <div style={{ marginBottom: '15px' }}>
      <label htmlFor={passwordId}>Password:</label><br />
      <input
        id={passwordId}
        name="password"
        type="password"
        value={formData.password}
        onChange={handleChange}
        placeholder="Enter your password"
        required
      />
    </div>
```

React Fragment

In React, a **Fragment** lets you **group multiple elements without adding extra nodes** to the DOM.

Why use <Fragment> or <>?

Normally, you can only return **one** element from a React component. But what if you want to return multiple sibling elements **without** wrapping them in a (which can mess up your layout)?

🤔 Jab dono se kaam chal raha hai, to Fragment hi kyun?

Farq #1: Extra HTML element nahi banta (DOM clean rehta hai)

** ** use karne se **extra element** create hota hai HTML structure mein.
<Fragment> use karne se **kuch bhi extra create nahi hota**, sirf grouping ke liye hota hai React ke andar.

Farq #2: Layout ya Styling bigad sakti hai

Kayi baar div add karne se **CSS styling ya layout toot jaata hai**.

Farq #3: Performance (chhota sa)

Jyada hone se browser ko unnecessary HTML render karna padta hai. Fragment use karne se ye avoid hota hai (though minor effect hai).

Farq #4: Semantic HTML maintain hoti hai

Aapka HTML structure semantically sahi rehta hai. SEO aur accessibility mein help karta hai.

Rules for React Hook

🔴 1. Only Call Hooks at the Top Level

🔴 2. Only Call Hooks from React Functions

🔁 3. Always Call Hooks in the Same Order

Making Custom Hook

ye ek simple javascript function he hota hai, usme hum react ke built in hooks ka use karte hai, aur jo bhi function ka naam hai uske strating mein **use** ho n chihiye, kyu ki hooks kis starting use se chalu hoti hai.

Because **custom hooks sirf JavaScript logic** handle karte hain — jaise useState, useEffect, useRef, etc. — aur **wo JSX (HTML jaisa code)** return **nahi** karte.

Normally **custom hook kabhi JSX return nahi karta.** Agar tum JSX return kar rahe ho, **wo component ban gaya**, hook nahi.

🔑 key in useLocalStorageMode(key = 'mode') ka matlab kya hai?

🔸 **localStorage ek key–value storage system hai**

Jab bhi tum kuch save karte ho localStorage me, **ek key aur ek value** dena padta hai.

js

CopyEdit

localStorage.setItem('mode', 'dark'); // 🔑 key = 'mode'

Toh "mode" yaha pe ek key hai, jiske andar 'dark' ya 'light' stored hota hai.

✅ key parameter ka fayda kya hai?

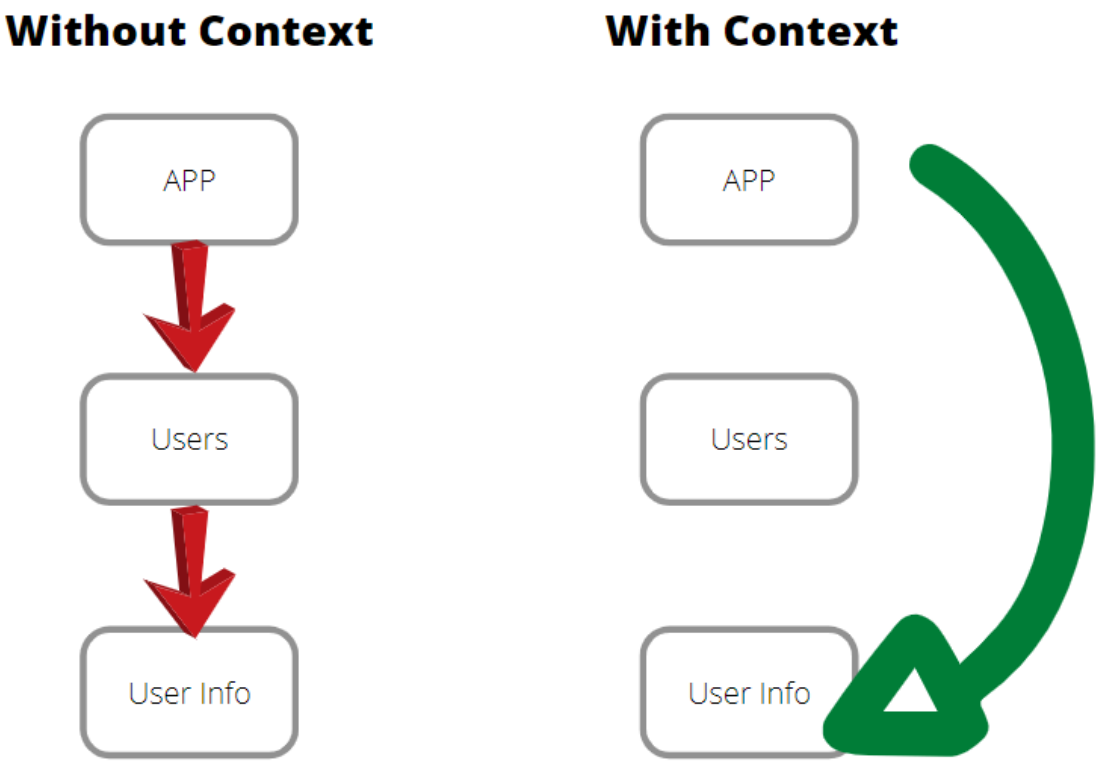
Agar tumhare app me multiple cheezein localStorage me store ho rahi hain, toh **har ek cheez ko alag key dena zaroori hai.**

Feature	Hardcoded Key Version	Key Parameter Version
Simplicity	✅ Simple	🔄 Thoda dynamic
Reusable for other keys?	❌ Not possible	✅ Yes (e.g., 'theme', 'sidebar')
Conflicts avoidable?	❌ No	✅ Yes
Ideal for big apps?	❌	✅

Context API

Agar tumhare paas React app mein **bahut saare components** hain, aur tumhe ek **data (jaise theme, user, language, etc.)** ko un **sabme share** karna hai — toh baar-baar props bhejna (props drilling) boring aur complex ho jata hai.

🟢 **Context API** ek aisa system hai jisse tum **ek hi jagah data define** kar ke **saare components ko easily access** karwa sakte ho — bina har baar props bhejne ke.



Props drilling nahi karna hai, isliye hum abb conext API use karenge.

Context API parts

- CreateContext → to initiate context Api
- Provider → use to update or provide data
- useContext → get data from context API

Kaise use karna hai

manalo abb jo bhi data hai woh App.jsx → A.jsx → B.jsx → C.jsx → D.jsx
app ka jo bhi data hai wo d tak jana chahiye

- **src** ke andar ek context ka folder banao jisme hum sab context bana sakte hai
- ek konse bhi naam ka jsx file banao

```
import { createContext } from "react";

export const UserContext = createContext();
```

- abb jo bhi parent hai jaise iss example mein app.jsx hai

```jsx
{/* App.jsx → A.jsx → B.jsx → C.jsx → D.jsx  */}
  <UserContext.Provider value={name}>
    <A />
  </UserContext.Provider>
```

- abb jo final wala hai jaise D.jsx wahape kya kre

```jsx
import React, { useContext } from 'react'
import { UserContext } from '../context/UserContext'

const D = () => {
    const name = useContext(UserContext);
  return (
    <div>
        Hello !! {name}
    </div>
  )
}

export default D
```
